國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab09

姓名：李品融

學號：113310217

Lab09-Q1
這題在練習二元搜尋樹 BST 的 deleteMode 遞迴函式，並加上詳細的註解。須要精確的處理刪除節點時的三種情境:無子節點、僅具單一子節點、擁有雙子節點時尋找右子樹最小值，並進行遞補的邏輯，確保刪除後依舊維持搜尋樹特性。

Code

```
#include <iostream>
using namespace std;

// ========================
// 樹的節點類別
// ========================
class TreeNode {
public:
 int data;           // 節點儲存的資料
 TreeNode* left;     // 指向左子節點
 TreeNode* right;    // 指向右子節點

 // 節點建構子：設定資料並初始化左右指標為空
 TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// ========================
// 二元搜尋樹類別
// ========================
class BinarySearchTree {
private:
 TreeNode* root; // 根節點指標

 // --- 私有的遞迴插入方法 ---
 TreeNode* insert(TreeNode* node, int val) {
  if (node == nullptr) {
   return new TreeNode(val); // 找到插入位置，建立新節點
  }

  if (val < node->data) {
   node->left = insert(node->left, val); // 插入左子樹
  }
```

```
 else if (val > node->data) {
  node->right = insert(node->right, val); // 插入右子樹
 }
 // 如果 val == node->data，不插入（避免重複）

 return node; // 回傳更新後的節點
}

// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
 if (node == nullptr) return false;

 if (node->data == target) return true;
 else if (target < node->data) return search(node->left, target); // 繼續在左子
樹找
 else return search(node->right, target);// 繼續在右子樹找
}

// --- 私有的中序遍歷方法（左-根-右）---
void inOrderTraversal(TreeNode* node) {
 if (node == nullptr) return;

 inOrderTraversal(node->left);   // 先走訪左子樹
 cout << node->data << " ";   // 印出目前節點的資料
 inOrderTraversal(node->right); // 再走訪右子樹
}

// --- 私有的刪除方法 ---
// node:目前正在處理的子樹根節點
// val:想要從樹中刪除的數值
TreeNode* deleteNode(TreeNode* node, int val) {
 // 第一步，確認終止條件
 // 如果該節點是空的，表示走到底也沒找到該數值，直接回傳 nullptr
 if (node == nullptr) return nullptr;

 // 第二步，尋找目標節點
 // 根據二元搜尋樹的特性進行查找
 if (val < node->data) {  // 如果目標值比目前節點小，往左子樹找
```

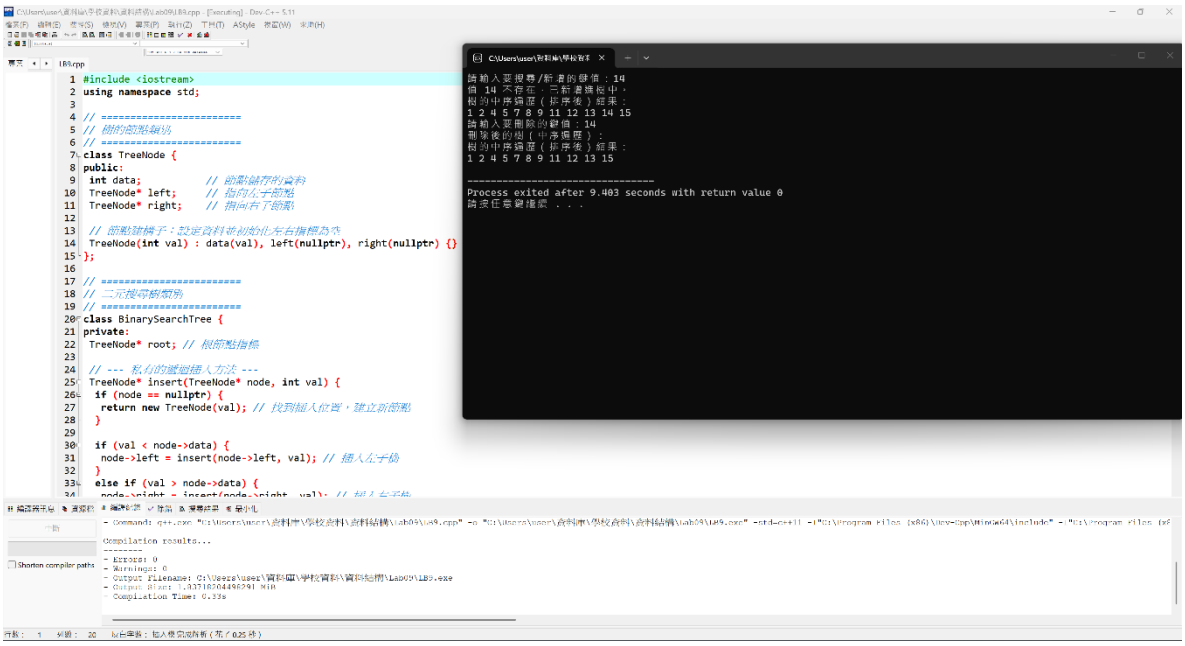
```
    node->left = deleteNode(node->left, val); // 刪除後，因為左子樹的根節點可能會變動，所以要重新賦值給 node->left
  }
  else if (val > node->data) { // 如果目標值比目前節點大，往右子樹找
    node->right = deleteNode(node->right, val); // 刪除後，因為右子樹的根節點可能會變動，所以要重新賦值給 node->right
  }
  else {
    // 第三步，找到目標節點，並執行刪除
    // 此時 val == node->data，這是要刪除的節點

    // 情況一、二:節點只有一個子節點或是葉子節點
    // 如果左邊沒人，就直接把右邊的子樹提拔上來
    // 如果右邊也是 nullptr，那麼 temp 就為 nullptr
    if (node->left == nullptr) {
      TreeNode* temp = node->right;
      delete node; // 釋放記憶體
      return temp; // 回傳右子樹，讓父節點接住
    }
    else if (node->right == nullptr) { // 如果右邊沒人，就直接把左邊的子樹提拔上來
      TreeNode* temp = node->left;
      delete node; // 釋放記憶體
      return temp; // 回傳左子樹，讓父節點接住
    }

    // 情況三:節點有兩個子節點
    // 為了不破壞結構， 需要找合適的繼承者來頂替目前的位置
    // 須尋找最接近目前節點但又比他大的數值，從右子樹找尋最小的節點
    TreeNode* temp = findMin(node->right); // 呼叫 findMin 找到右子樹的最小值節點

    node->data = temp->data; // 將繼承者的數值複製到當前節點

    // 透過遞迴，刪除右子樹中被複製過來的節點
    node->right = deleteNode(node->right, temp->data);
  }
  return node; // 回傳更新後的子樹根節點
}
```

```
 // --- 私有的找最小值方法 ---
 TreeNode* findMin(TreeNode* node) {
  while (node->left != nullptr) {
   node = node->left;
  }
  return node;
 }

public:
 // --- 建構子：初始化為空樹 ---
 BinarySearchTree() : root(nullptr) {}

 // --- 公開插入方法 ---
 void insert(int val) {
  root = insert(root, val);
 }

 // --- 公開搜尋方法 ---
 bool search(int target) {
  return search(root, target);
 }

 // --- 公開中序遍歷方法 ---
 void displayInOrder() {
  cout << "樹的中序遍歷（排序後）結果：" << endl;
  inOrderTraversal(root);
   cout << endl;
 }

 // --- 公開刪除方法 ---
 void deleteValue(int val) {
  root = deleteNode(root, val);
 }
};

int main() {
 BinarySearchTree bst; // 建立一棵空的樹
```

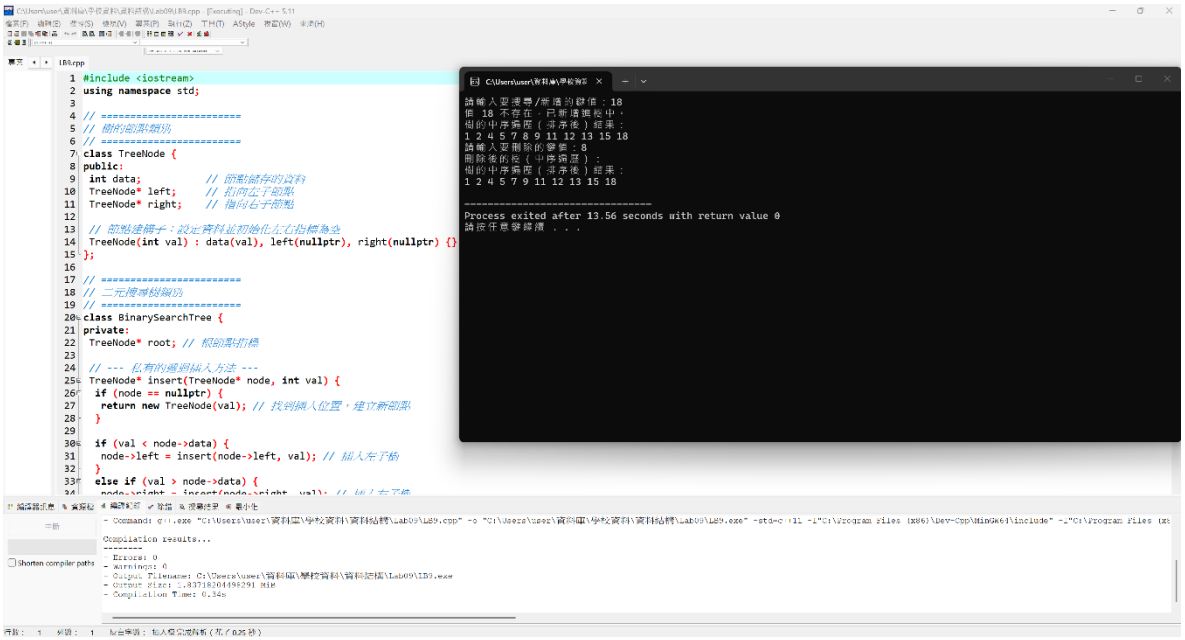
```
// 預設插入的資料
int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
int n = sizeof(values) / sizeof(values[0]);

// 把資料插入樹中
for (int i = 0; i < n; ++i) {
 bst.insert(values[i]);
}

// 讓使用者輸入要插入的鍵值
int key;
cout << "請輸入要搜尋/新增的鍵值：";
cin >> key;

if (bst.search(key)) {
 cout << "值 " << key << " 已存在樹中，不新增。" << endl;
}
else {
 cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
 bst.insert(key);
}

// 顯示目前樹的中序遍歷結果
bst.displayInOrder();

// 讓使用者輸入要刪除的鍵值
int deleteKey;
cout << "請輸入要刪除的鍵值：";
cin >> deleteKey;

// 刪除節點
bst.deleteValue(deleteKey);

// 顯示刪除後的樹
cout << "刪除後的樹（中序遍歷）：" << endl;
bst.displayInOrder();
return 0;
}
```

Result

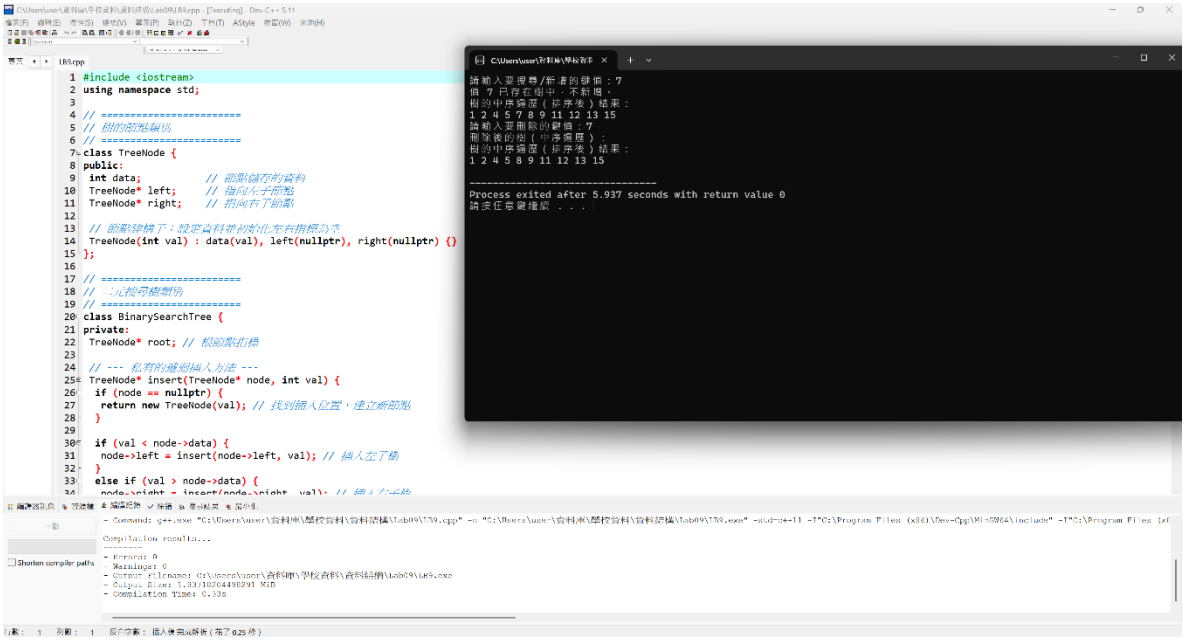
Test-1:輸入[14]，刪除[14]



Test-2:輸入[18]，刪除[8]



Test-3:輸入[7]，刪除[7]



```
#include <iostream>
using namespace std;

// ========================
// 樹的節點類別
// ========================
class TreeNode {
public:
 int data;           // 節點儲存的資料
 TreeNode* left;     // 指向左子節點
 TreeNode* right;    // 指向右子節點

 // 節點建構子：接受資料並初始化左右指標為空
 TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// ========================
// 二元搜尋樹類別
// ========================
class BinarySearchTree {
private:
 TreeNode* root; // 根節點指標

 // --- 私有的遞迴插入方法 ---
 TreeNode* insert(TreeNode* node, int val) {
  if (node == nullptr) {
   return new TreeNode(val); // 找到插入位置，建立新節點
  }

  if (val < node->data) {
   node->left = insert(node->left, val); // 插入左子樹
  }
  else if (val > node->data) {
   node->right = insert(node->right, val); // 插入右子樹
```

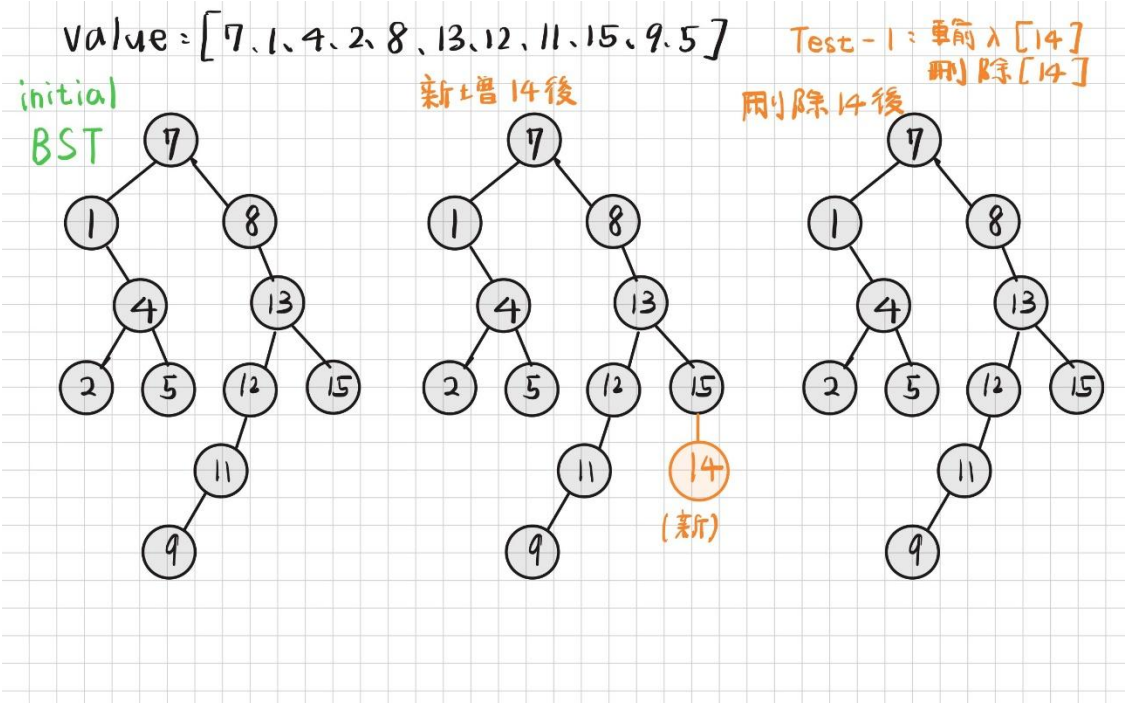
```
請輸入要搜尋/新增的數值：7
值 7 已存在樹中，不新增。
樹的中序遍歷（排序後）結果：
1 2 4 5 7 8 9 11 12 13 15
請輸入要刪除的數值：7
刪除後的樹（中序遍歷）：
樹的中序遍歷（排序後）結果：
1 2 4 5 8 9 11 12 13 15

--------------------------------
Process exited after 5.937 seconds with return value 0
請按任意鍵繼續 . . .
```

Lab09-Q3

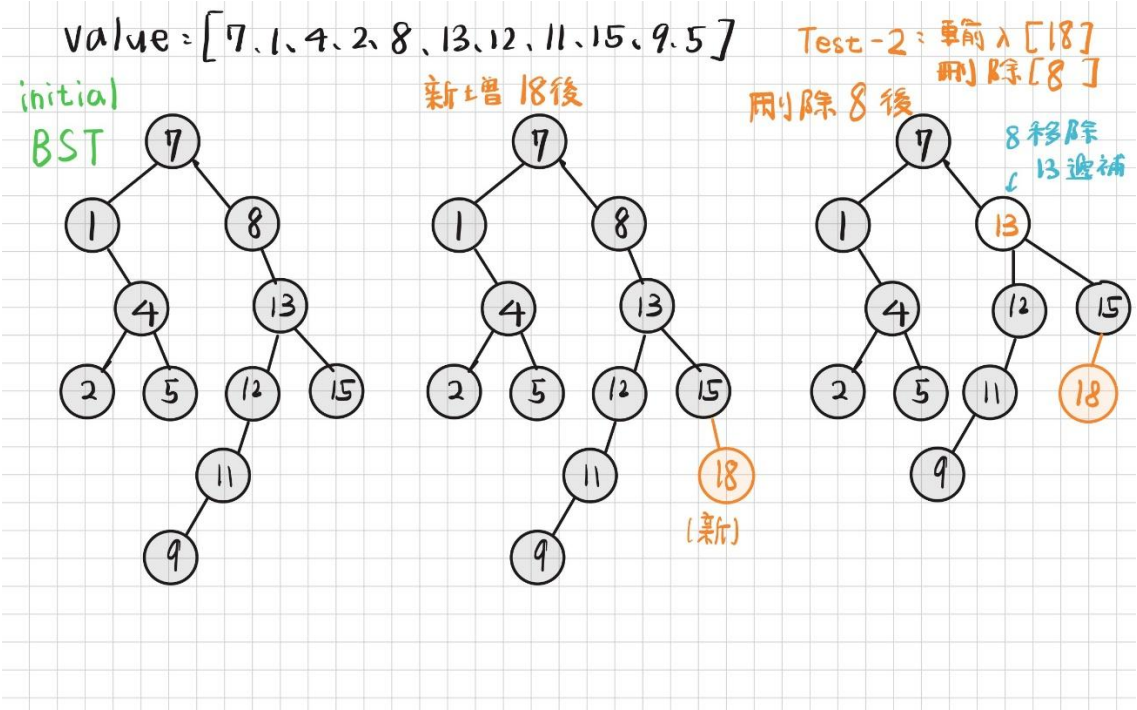
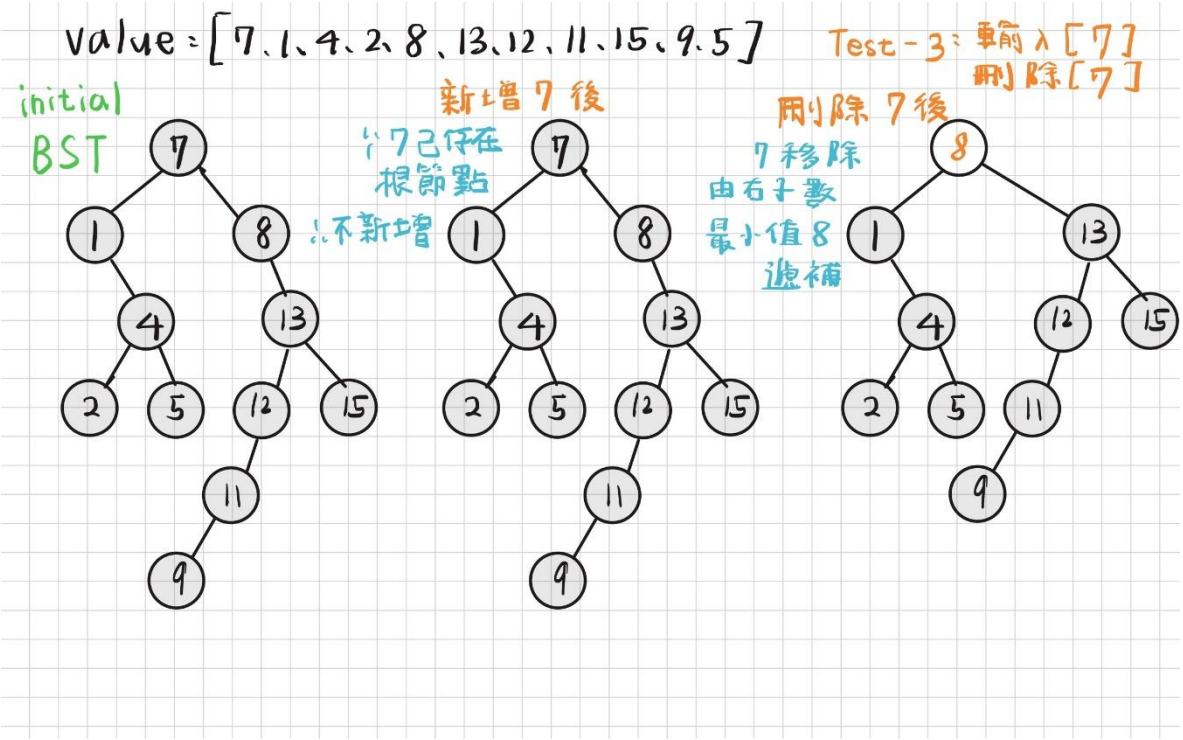
這題在練習繪製樹狀圖。透過追蹤數值的插入與刪除，來繪製出節點新增至葉端的路徑，以及刪除根節點的樹狀圖，達到視覺化呈現 BST 的動態演變。

Result

Test-1:輸入[14]，刪除[14]



Test-2:輸入[18]，刪除[8]



Test-3:輸入[7]，刪除[7]